

”

E-fólio A | Folha de resolução para E-fólio



UNIDADE CURRICULAR: Compilação

CÓDIGO: 21018

Grupo: DUALCORE

João Rodrigues (2203474)

Maria Costa (2304361)

CURSO: Licenciatura em Engenharia Informática

DATA DE ENTREGA: 20/04/2026

1. Introdução

O presente trabalho tem como objetivo implementar, com recurso ao ANTLR4 e Python, o processamento da linguagem MOCP (My Own C in Português), de acordo com o enunciado do E-Fólio A. O foco principal da solução incide na análise léxica, análise sintática e construção de uma árvore sintática abstrata (AST), lendo o programa a partir de um ficheiro e apresentando os erros detetados no ecrã.

A linguagem MOCP corresponde a uma versão simplificada da linguagem C, mas com palavras-chave, tipos e funções reservadas em português. Entre as restrições relevantes incluem-se a inexistência de diretivas como `#include`, a ausência de estruturas, a utilização de apenas dois tipos base (`inteiro` e `real`), a obrigatoriedade de blocos com chavetas nas estruturas de controlo e a proibição de operadores como `++`, `--`, `+=` e semelhantes.

2. Opções de implementação

A ferramenta escolhida foi o ANTLR4, com geração de lexer, parser, visitor e listener para Python. Esta opção permitiu definir a componente léxica e sintática num único ficheiro de gramática (`MOCP.g4`) e obter automaticamente a infraestrutura necessária para o processamento do código-fonte.

A solução foi organizada de forma modular, separando claramente:

- a gramática (`MOCP.g4`);
- o programa principal (`main.py`);
- os nós da AST (`ast_nodes.py`);
- a construção da AST (`ast_builder.py`);
- o tratamento melhorado de erros (`error_handler.py`);
- a validação semântica básica adicional (`semantic.py`).

Esta organização facilita a leitura, a manutenção e a evolução futura do projeto.

3. Análise léxica

A análise léxica foi implementada através das regras lexer da gramática ANTLR. Foram definidos tokens para palavras-chave da linguagem, funções reservadas de entrada/saída,

literais, identificadores, operadores e delimitadores, sendo comentários e espaços ignorados.

Foi ainda introduzida a detecção de palavras-chave e operadores da linguagem C não suportados em MOCP (como `int`, `double`, `if`, `++`, `+=`). O módulo `error_handler.py` percorre os tokens e apresenta mensagens claras com sugestões corretas. Por exemplo:

```
[LÉXICO/SINTÁTICO] linha 14, coluna 0: Palavra-chave C 'int'
detetada.                Em                MOCP                use                'inteiro'.
[LÉXICO/SINTÁTICO] linha 22, coluna 5: Operador '++' não é suportado
em MOCP. Use i = i + 1.
```

Esta abordagem melhora significativamente a qualidade do diagnóstico.

4. Análise sintática

A gramática foi definida de forma a refletir a estrutura da linguagem MOCP. O programa é composto por protótipos, declarações globais e definições de funções.

Foi adotada a separação entre:

- `returnType` (inclui vazio);
- `varType` (apenas inteiro e real);
- `castType` (apenas inteiro e real).

Esta distinção permite rejeitar construções inválidas diretamente ao nível sintático.

4.1 Declarações de variáveis

A gramática suporta diferentes formas de declaração através de alternativas rotuladas:

`varItem`

```
: ID                                     # VarSimpleDecl
| ID ASSIGN expr                         # VarInitDecl
| ID LBRACK INT_LITERAL RBRACK          # VarSizedArrayDecl
| ID LBRACK RBRACK ASSIGN arrayInit      # VarArrayInitDecl
| ID LBRACK RBRACK ASSIGN expr           # VarArrayExprInitDecl
| ID LBRACK RBRACK                       # VarUnSizedArrayDecl
;
```

Exemplos:

- `inteiro x;`
- `inteiro a[] = {1,2,3};`

4.2 Precedência de operadores

As expressões são definidas por uma hierarquia de regras que garante a precedência correta entre operadores. O padrão $(a \text{ op } b) *$ assegura associatividade à esquerda, sendo garantido na AST que:

$$a - b - c \equiv (a - b) - c$$

4.3 Condições

As condições são tratadas separadamente das expressões aritméticas, garantindo precedência correta dos operadores lógicos (`!` `>` `&&` `>` `||`) e evitando ambiguidades.

4.4 O problema do Dangling Else

O problema do *dangling else* não ocorre na gramática MOCP, uma vez que a instrução se exige sempre blocos `{}`:

```
ifStmt : SE LPAREN condExpr RPAREN block (SENAO block)? ;
```

Esta decisão elimina ambiguidades por construção.

5. Construção da AST

Foi utilizado o padrão Visitor do ANTLR para construir uma AST, permitindo separar a parse tree de uma representação mais abstrata.

A AST inclui nós para programa, funções, blocos, declarações, instruções e expressões. A sua impressão é feita através do método `pretty()`, permitindo visualizar a estrutura do programa de forma hierárquica.

Esta representação facilita futuras extensões, como análise semântica mais avançada ou geração de código.

6. Validação semântica básica adicional

Apesar de não ser exigida, foi implementada uma validação semântica básica, incluindo:

- utilização de variáveis não declaradas;
- coerência entre protótipos e funções;
- detecção de funções declaradas mas não definidas;
- validação básica de retornos e chamadas.

Esta componente é complementar e não constitui uma análise semântica completa.

Apesar de funcional, a solução apresenta limitações, nomeadamente a ausência de uma análise semântica aprofundada e de geração de código, constituindo possíveis evoluções futuras.

7. Dificuldades encontradas

As principais dificuldades incluíram:

- tratamento da palavra-chave `principal` como identificador especial;
- ordenação correta dos tokens para evitar conflitos lexicais;
- separação entre tipos (`returnType`, `varType`, `castType`);
- garantia de associatividade à esquerda na construção da AST.

8. Conclusão

O trabalho desenvolvido permite processar programas em MOCP com análise léxica e sintática corretas, construção de uma AST coerente e apresentação de erros ao utilizador. A solução evidencia uma gramática rigorosa, uma arquitetura modular e uma abordagem consistente ao processamento da linguagem, constituindo uma base sólida para evolução futura no contexto da construção de compiladores.